Around the Globe Quiz

Python Project



Presented by:
Laura Sarkulova

About the Project

The project goal was create an interactive, fully authomated Python application that automatically generates geography multiple-choice quizzes using real-world country data using datasets.







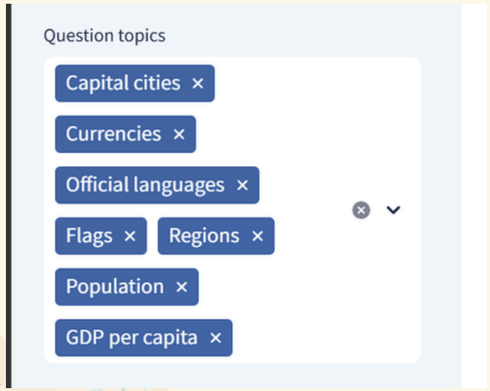
Main objectives

1. Create questions covering geographical facts and country statistics.
2. Include four answer choices for every question.
3. Make sure each question has only one correct answer.
4. Vary the similarity of incorrect options based on the chosen difficulty level.
5. Enable players to answer the generated questions.
6. Evaluate player performance using a score weighted by question difficulty.

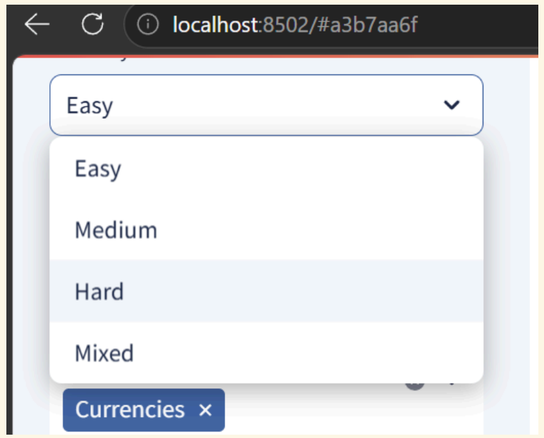
Settings

Implemented question categories

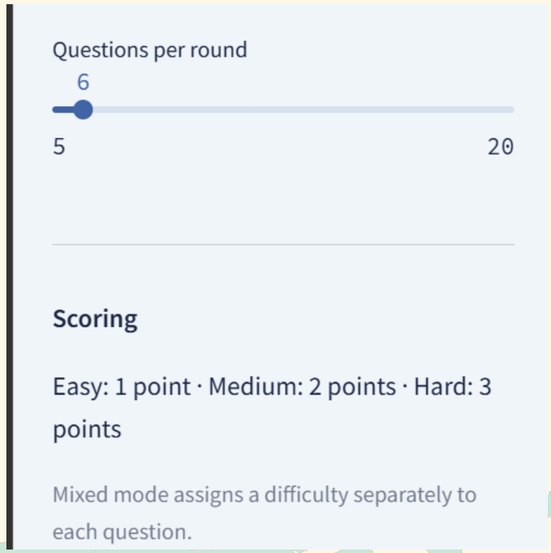
- Capital cities
- Flags
- Official languages
- Currencies
- Regions
- Population
- GDP per capita



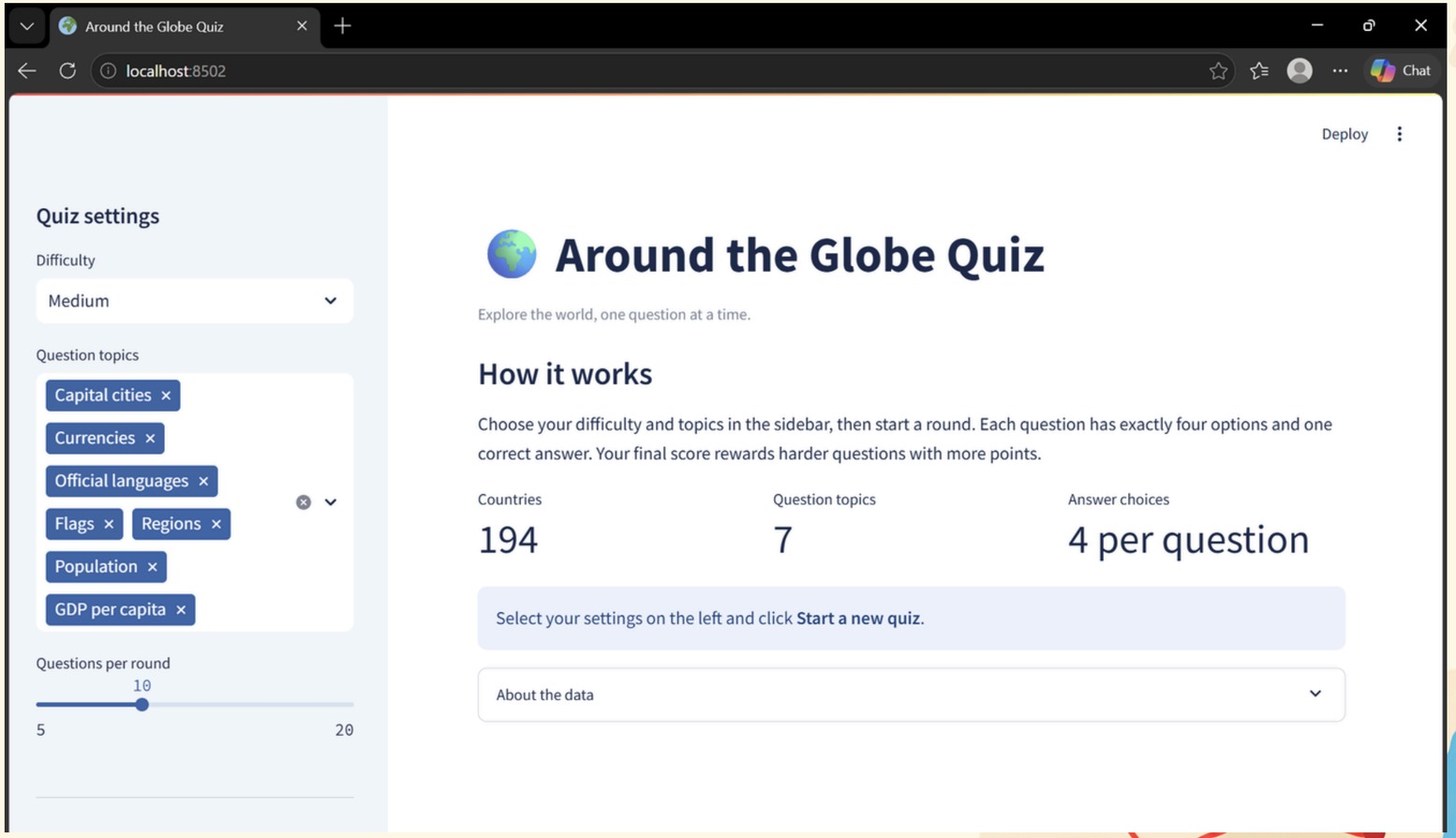
4 difficulty levels:
Easy
Medium
Hard
Mixed



Questions per round
1–10



Interface

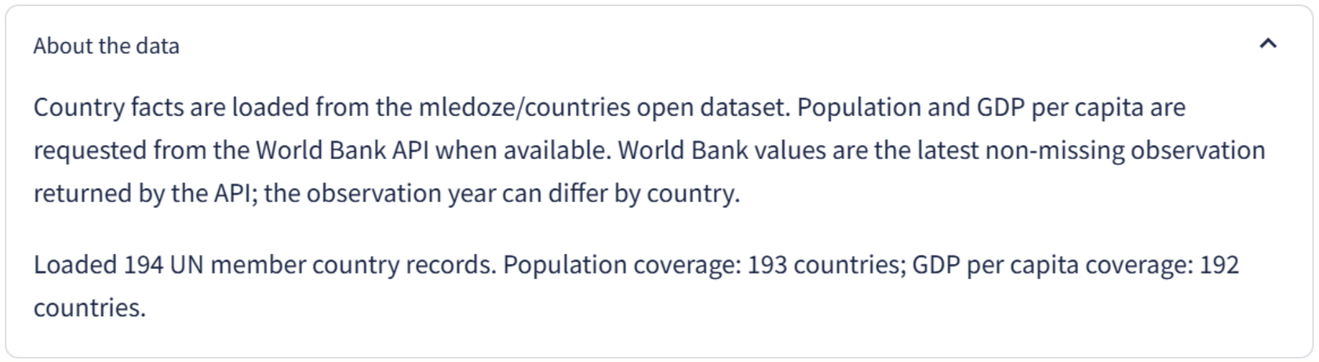

Around the Globe Quiz
localhost:8502
Chat
Deploy
Quiz settings
Difficulty
Medium
Question topics
Capital cities ×
Currencies ×
Official languages ×
Flags ×
Regions ×
Population ×
GDP per capita ×
Questions per round
10
5
20
Around the Globe Quiz
Explore the world, one question at a time.
How it works
Choose your difficulty and topics in the sidebar, then start a round. Each question has exactly four options and one correct answer. Your final score rewards harder questions with more points.
Countries
194
Question topics
7
Answer choices
4 per question
Select your settings on the left and click Start a new quiz.
About the data

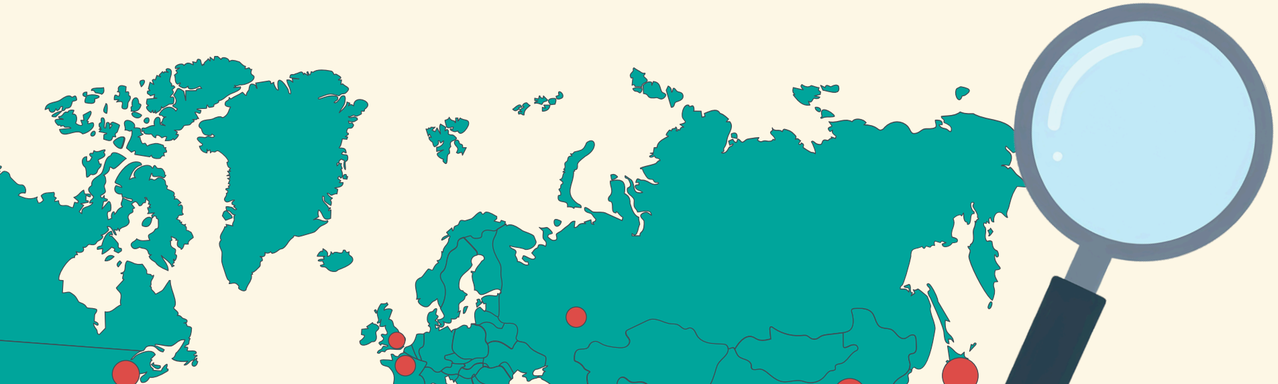




Data Sources

The project uses two main external data sources.





01

REST Countries dataset

URL: https://github.com/mledoze/countries

Provides:

- Country names
- Capital cities
- Flags
- Currencies
- Official languages
- Regions and geographical information

02

World Bank API

URL: https://datahelpdesk.worldbank.org/knowledgebase/articles/889392-about-the-indicators-api-documentation

Provides:

- Population
- GDP per capita

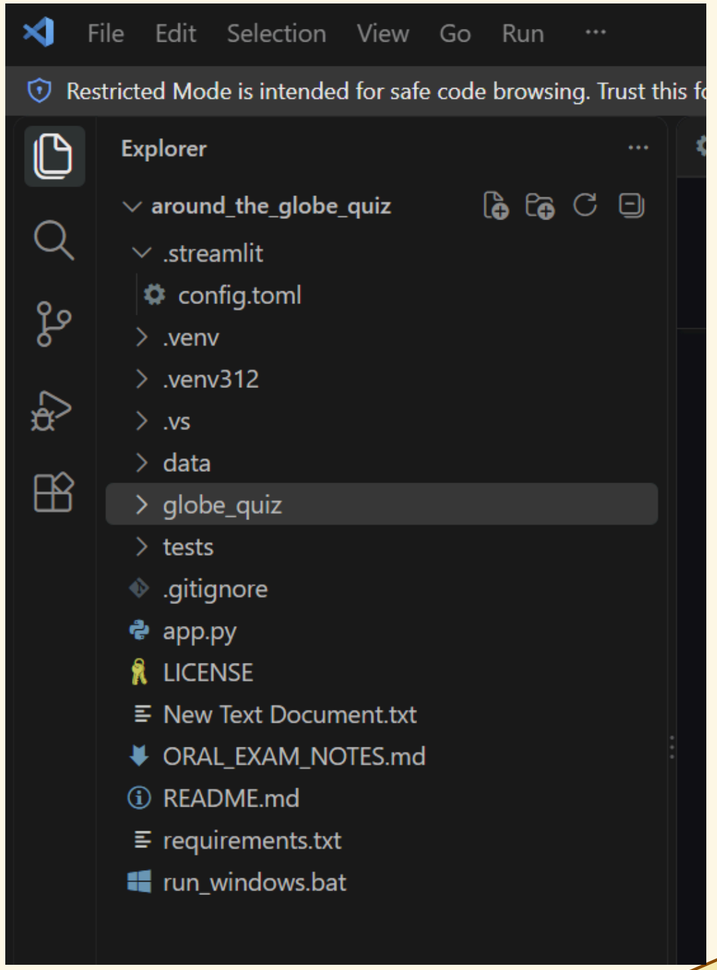
Data Collection and Processing

1. Python Requests retrieves external data.
2. JSON is used as the original data format.
3. Pandas cleans and organizes the information.
4. The datasets are combined into a structured DataFrame.
5. Local caching helps avoid unnecessary repeated downloads.





Project Structure



Main technologies

- Python — main programming language.
- Streamlit — interactive web application.
- Pandas — data manipulation.
- NumPy — numerical calculations.
- Matplotlib — data visualization.
- Requests — external data retrieval.
- Pytest — automated testing.

app.py
Streamlit interface and application flow

globe_quiz/data.py
Downloading, cleaning, and preparing country data

globe_quiz/quiz.py
Generating questions and calculating scores

globe_quiz/analysis.py
Analyzing results and creating visualizations

tests/
Automated tests

requirements.txt
Project dependencies

README.md
Project documentation and details





```python
def load_country_data(refresh: bool = False, include_world_bank: bool = True) -> pd.DataFrame:
    """Load and combine country facts with optional World Bank indicators."""
    countries = prepare_countries(load_raw_countries(refresh=refresh))

    if include_world_bank:
        for field, indicator in INDICATORS.items():
            extra = load_world_bank_indicator(indicator)
            extra = extra.rename(columns={indicator: field})

            if not extra.empty:
                countries = countries.merge(extra, on="cca3", how="left")
            else:
                countries[field] = pd.NA
    else:
        countries["population"] = pd.NA
        countries["gdp_per_capita"] = pd.NA
```

This function loads information from external APIs and combines the country data with World Bank statistics. I use pandas to merge and prepare the data for the quiz.

This is the main function of my project. It generates questions dynamically using country data. It selects one correct answer and three incorrect answers. The difficulty level determines how similar the incorrect answers are to the correct one.

```python


def generate_question(countries: pd.DataFrame, difficulty: str = "Medium",
                      category: str | None = None, rng: random.Random | None = None) -> Question:
    """Generate one valid four-option geography question.

    Easy questions use random distractors. Medium questions prefer somewhat
    similar countries. Hard questions select the most similar available
    distractors (for population, the closest values on a logarithmic scale).
    """
    if difficulty not in DIFFICULTY_POINTS:
        raise ValueError(f"Unknown difficulty: {difficulty}")
    rng = rng or random.Random()
    if category is None:
        category = rng.choice(list(QUESTION_TYPES.values()))
```

```python


def _similarity_scores(target: pd.Series, candidates: pd.DataFrame, category: str) -> pd.Series:
    """Score candidate answers by similarity; higher means a harder distractor."""
    scores = pd.Series(0.0, index=candidates.index)
    if category in {"capital", "currency", "languages", "region"}:
        if target.get("region"):
            scores += (candidates["region"] == target["region"]).astype(float) * 2.0
        if target.get("subregion"):
            scores += (candidates["subregion"] == target["subregion"]).astype(float) * 2.0
        if category == "languages" and target.get("languages"):
            target_words = set(str(target["languages"]).split(", "))
            scores += candidates["languages"].fillna("").apply(
                lambda text: bool(target_words.intersection(set(str(text).split(", "))))
            ).astype(float) * 3.0
        if category == "currency" and target.get("currency"):
            target_words = set(str(target["currency"]).split(", "))
            scores += candidates["currency"].fillna("").apply(
```

This function calculates how similar other countries are to the selected country. For example, it considers shared regions, languages and currencies. For population and GDP, it uses logarithmic distance. This helps the application select more challenging incorrect answers.

This function calculates the final score. Each difficulty level has a different weight: Easy gives one point, Medium gives two points and Hard gives three points. NumPy is used to calculate the total earned points and the final percentage.

```python


def calculate_score(results: list[dict[str, Any]]) -> dict[str, float | int]:
    """Calculate a difficulty-weighted score as a percentage of available points."""
    if not results:
        return {"earned_points": 0, "maximum_points": 0, "score_percent": 0.0, "correct_answers": 0}
    weights = np.array([DIFFICULTY_POINTS[item["difficulty"]] for item in results], dtype=float)
    correct = np.array([bool(item["is_correct"]) for item in results], dtype=bool)
    earned = float(np.sum(weights * correct))
    maximum = float(np.sum(weights))
    return {
        "earned_points": int(earned),
        "maximum_points": int(maximum),
        "score_percent": round(100 * earned / maximum, 1) if maximum else 0.0,
        "correct_answers": int(correct.sum()),
    }


```

Activating the Python virtual environment. Installing Streamlit, pandas, NumPy, Matplotlib, and Requests libraries.

```
https://www.canva.com/design/DAHaVhffpq8/NUAIQxMrmps-1ECuzTLk8g/edit

C:\Windows\System32\cmd.e

C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>python --version
Python install manager was successfully updated to 26.4.

Please see https://github.com/python/pymanager/releases/tag/26.4 for all changes.

********************************************************************************
The signature for https://www.python.org/ftp/python/index-windows.json was successfully verified.
Installing Python 3.15.2.
Downloading: ..........................................................................✅
Extracting: ...........................................................................✅
To see all available commands, run 'py help'
********************************************************************************
Python 3.15.2

C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>py -m venv .venv

C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>.venv\Scripts\activate

(.venv) C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>py -m pip install -r requirements.txt
Collecting streamlit==1.41.1 (from -r requirements.txt (line 1))
  Downloading streamlit-1.41.1-py2.py3-none-any.whl.metadata (8.5 kB)
Collecting pandas==2.2.3 (from -r requirements.txt (line 2))
  Downloading pandas-2.2.3.tar.gz (4.4 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.4/4.4 MB 105.8 kB/s  0:00:38
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... error
  error: subprocess-exited-with-error
```

```
Downloading jsonschema_specifications-2025.9.1-py3-none-any.whl (18 kB)
Downloading markupsafe-3.0.4-cp312-cp312-win_amd64.whl (14 kB)
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Downloading referencing-0.37.0-py3-none-any.whl (26 kB)
Downloading rpds_py-2026.9.1-cp312-cp312-win_amd64.whl (222 kB)
Downloading smmap-5.0.3-py3-none-any.whl (24 kB)
Installing collected packages: pytz, watchdog, urllib3, tzdata, typing-extensions, tornado, toml, tenacity, smmap, rpds-
py, pyparsing, pygments, pyarrow, protobuf, pluggy, pillow, packaging, numpy, narwhals, mdurl, MarkupSafe, kiwisolver, i
niconfig, idna, fonttools, cycler, colorama, click, charset-normalizer, certifi, cachetools, blinker, attrs, requests, r
eferencing, python-dateutil, pytest, markdown-it-py, jinja2, gitdb, contourpy, rich, pydeck, pandas, matplotlib, jsonsch
ema-specifications, gitpython, jsonschema, altair, streamlit
Successfully installed MarkupSafe-3.0.4 altair-5.5.0 attrs-26.1.0 blinker-1.9.0 cachetools-5.5.2 certifi-2026.10.23 char
set-normalizer-3.5.2 click-8.5.1 colorama-0.4.6 contourpy-1.4.0 cycler-0.12.1 fonttools-4.68.0 gitdb-4.0.12 gitpython-3.
2.1 idna-3.20 iniconfig-2.3.1 jinja2-3.1.6 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 kiwisolver-1.5.1 markdow
n-it-py-4.2.0 matplotlib-3.10.0 mdurl-0.1.2 narwhals-2.28.0 numpy-2.2.1 packaging-24.2 pandas-2.2.3 pillow-11.3.0 pluggy
-1.7.0 protobuf-5.29.6 pyarrow-26.0.1 pydeck-0.9.3 pygments-2.21.0 pyparsing-3.3.3 pytest-8.3.4 python-dateutil-4.0.0 py
tz-2026.6 referencing-0.37.0 requests-2.32.3 rich-13.9.4 rpds-py-2026.9.1 smmap-5.0.3 streamlit-1.41.1 tenacity-9.2.1 to
ml-0.10.2 tornado-6.6 typing-extensions-4.16.0 tzdata-2026.6 urllib3-2.8.0 watchdog-6.0.0

(.venv312) C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>python -m streamlit run app.py

     Welcome to Streamlit!

     If you'd like to receive helpful onboarding emails, news, offers, promotions,
     and the occasional swag, please enter your email address below. Otherwise,
     leave this field blank.

     Email:

  You can find our privacy policy at https://streamlit.io/privacy-policy
```

Run the Streamlit application.

```
C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>.venv312\Scripts\activate

(.venv312) C:\Users\Laura\Downloads\around_the_globe_quiz\around_the_globe_quiz>python -m streamlit run app.py

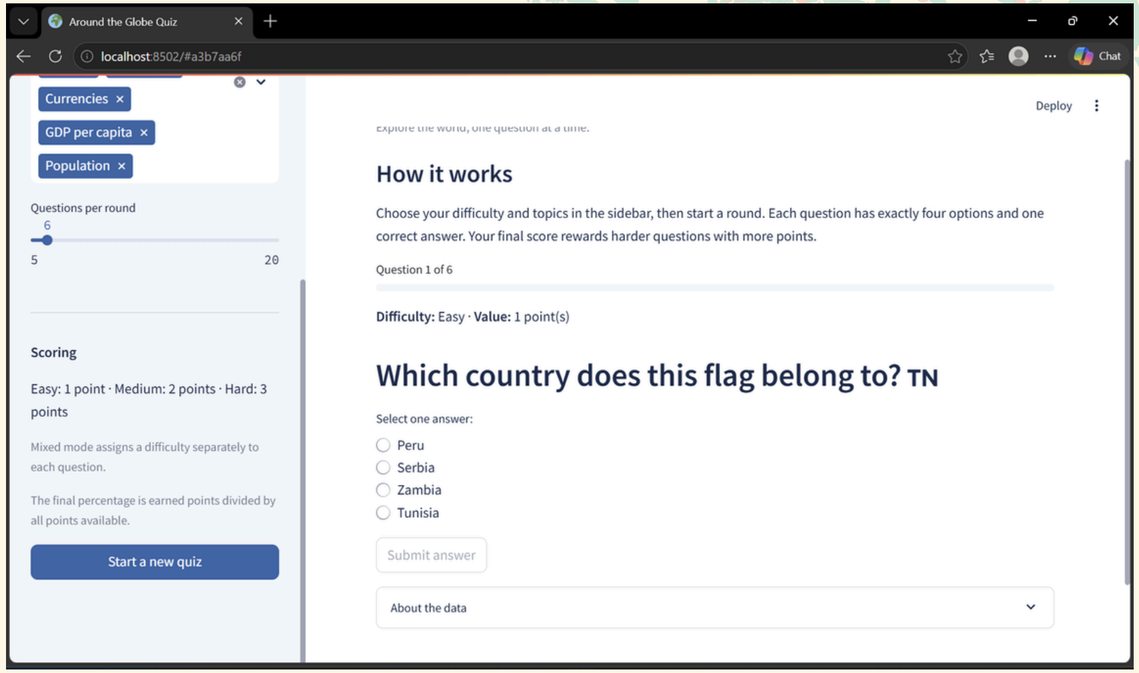
  You can now view your Streamlit app in your browser.

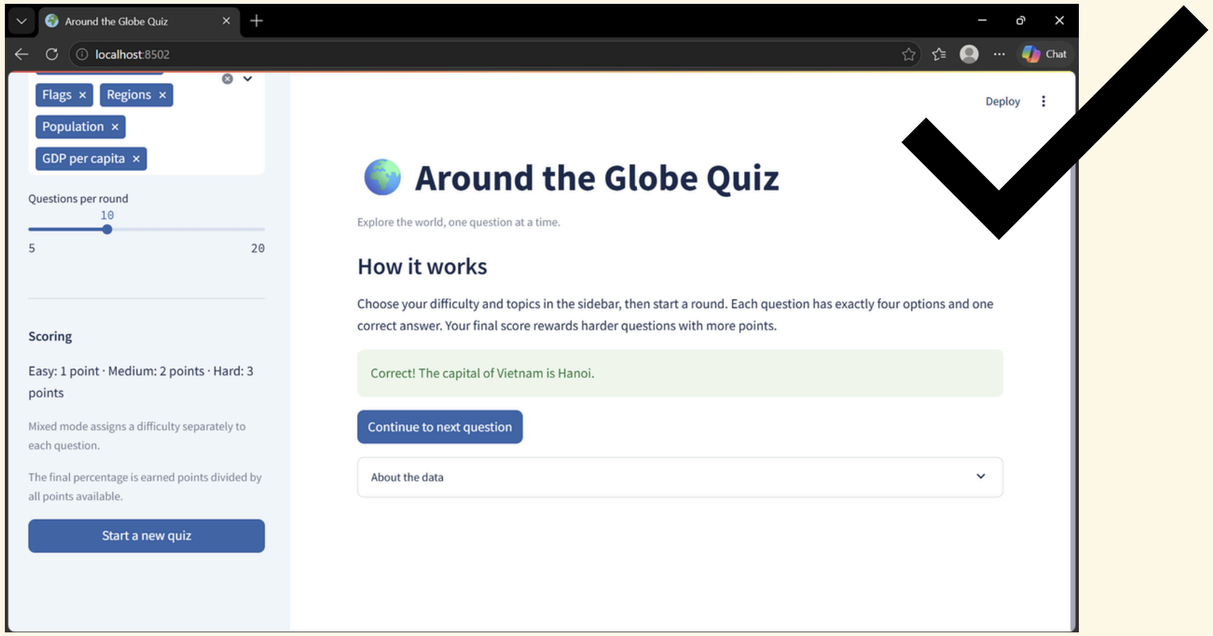
  Local URL: http://localhost:8502
  Network URL: http://172.20.10.6:8502
```

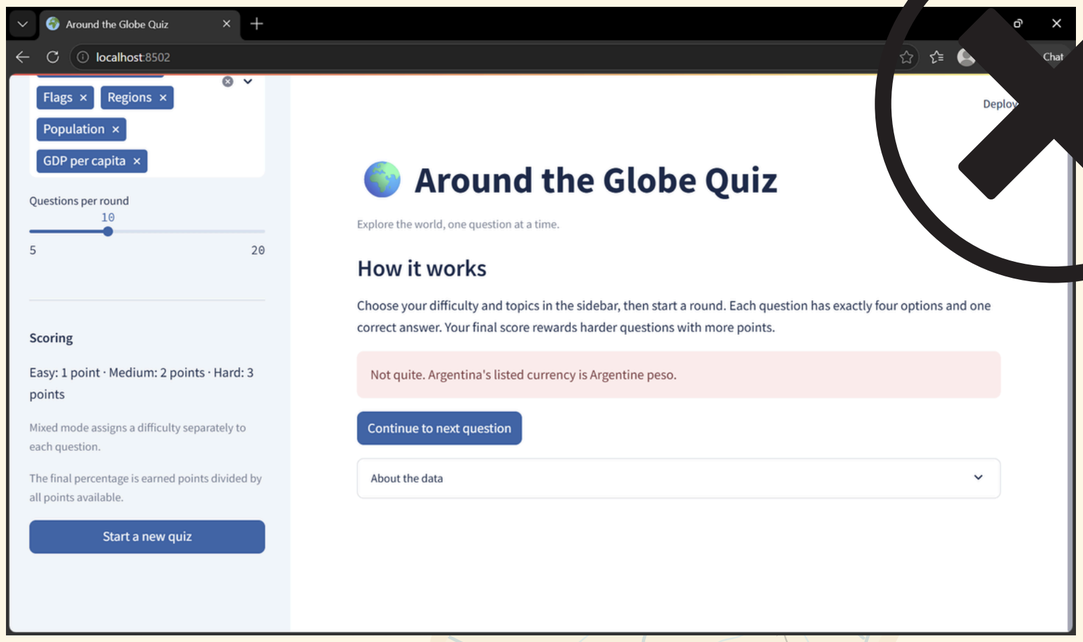


Starting the quiz

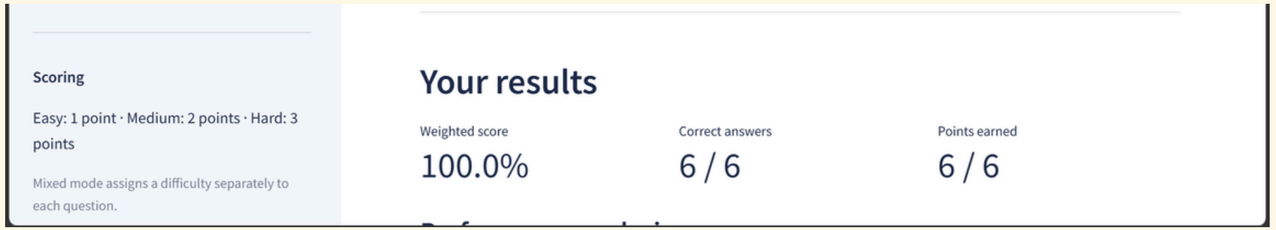
Questions are generated dynamically using the country dataset. They are not taken from a predefined question bank and are not generated by artificial intelligence.







Results



Pandas — data organization.
NumPy — calculations.
Matplotlib — visualizations.

After the quiz, the application calculates the final score and analyzes the player's performance. Pandas, NumPy, and Matplotlib are used to process results and create visualizations.

